

Examen d'Algorithmie et Programmation Parallèle

Marc Pérache

Corrigé

Durée : 2 heures
Tous documents papiers autorisés

Instructions

- Lisez attentivement toutes les questions avant de commencer.
- Soyez clair et concis dans vos réponses.
- Le barème est donné à titre indicatif.

1 Questions de Cours (20 points)

1. Qu'est-ce que MPI (Message Passing Interface)? Donnez un exemple d'utilisation simple. (4 points)

Correction :

MPI est une bibliothèque permettant la communication entre processus dans un environnement parallèle. Exemple simple : envoyer un message d'un processus à un autre.

2. Expliquez ce qu'est un communicateur en MPI. Donnez un exemple de création et d'utilisation d'un communicateur. (4 points)

Correction :

— Explication d'un Communicateur en MPI :

- En MPI (Message Passing Interface), un communicateur est un objet qui définit un groupe de processus pouvant communiquer entre eux.
- Il permet de définir des sous-ensembles de processus qui peuvent échanger des messages de manière isolée des autres processus.
- Les communicateurs sont essentiels pour structurer les communications dans les applications parallèles.

— Exemple de Création et d'Utilisation d'un Communicateur :

- Voici un exemple simple en C utilisant MPI pour créer et utiliser un communicateur.

```
1 #include <mpi.h>
2 #include <stdio.h>
3
4 int main(int argc, char *argv[]) {
5     MPI_Init(&argc, &argv);
6
7     int world_rank, world_size;
```

```

8     MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &world_size);
10
11    // Création d'un nouveau communicateur pour les processus
    de rang pair
12    MPI_Comm new_comm;
13    MPI_Comm_split(MPI_COMM_WORLD, world_rank % 2, world_rank,
        &new_comm);
14
15    int new_rank, new_size;
16    MPI_Comm_rank(new_comm, &new_rank);
17    MPI_Comm_size(new_comm, &new_size);
18
19    printf("Processus global %d sur %d - Processus local %d
    sur %d\n",
20           world_rank, world_size, new_rank, new_size);
21
22    // Utilisation du nouveau communicateur pour une
    communication
23    if (new_rank == 0) {
24        int message = 42;
25        MPI_Send(&message, 1, MPI_INT, 1, 0, new_comm);
26        printf("Processus %d a envoyé le message %d\n",
    new_rank, message);
27    } else if (new_rank == 1) {
28        int message;
29        MPI_Recv(&message, 1, MPI_INT, 0, 0, new_comm,
    MPI_STATUS_IGNORE);
30        printf("Processus %d a reçu le message %d\n", new_rank
    , message);
31    }
32
33    MPI_Comm_free(&new_comm);
34    MPI_Finalize();
35    return 0;
36 }
37

```

— Explication du Code :

- **Initialisation de MPI :** MPI_Init initialise l'environnement MPI.
- **Obtention du Rang et de la Taille du Communicateur Global :** MPI_Comm_rank et MPI_Comm_size obtiennent le rang et la taille du communicateur global MPI_COMM_WORLD.
- **Création d'un Nouveau Communicateur :** MPI_Comm_split crée un nouveau communicateur en divisant MPI_COMM_WORLD en deux groupes : les processus de rang pair et les processus de rang impair.
- **Obtention du Rang et de la Taille du Nouveau Communicateur :** MPI_Comm_rank et MPI_Comm_size obtiennent le rang et la taille du nouveau communicateur.
- **Communication à l'Intérieur du Nouveau Communicateur :** Les processus de rang 0 et 1 dans le nouveau communicateur échangent un message.
- **Libération du Communicateur :** MPI_Comm_free libère le nouveau communicateur.
- **Finalisation de MPI :** MPI_Finalize termine l'environnement MPI.

— Conclusion :

- Les communicateurs en MPI permettent de structurer les communications entre les processus de manière flexible et efficace.

- Ils sont essentiels pour développer des applications parallèles complexes où différents groupes de processus doivent communiquer de manière isolée.

3. Quelle est la différence entre les communications bloquantes et non-bloquantes en MPI? Donnez un exemple de chaque type de communication. (4 points)

Correction :

— **Communications bloquantes :**

- Dans une communication bloquante, l'appel de la fonction de communication ne retourne pas tant que l'opération n'est pas terminée.
- Cela signifie que le processus appelant est bloqué jusqu'à ce que la communication soit complète.
- Exemple : MPI_Send et MPI_Recv
 - MPI_Send(buffer, count, datatype, dest, tag, comm)
 - MPI_Recv(buffer, count, datatype, source, tag, comm, status)

— **Communications non-bloquantes :**

- Dans une communication non-bloquante, l'appel de la fonction de communication retourne immédiatement, permettant au processus de continuer son exécution.
- L'opération de communication se fait en arrière-plan et doit être complétée explicitement par une autre fonction.
- Exemple : MPI_Isend et MPI_Irecv
 - MPI_Isend(buffer, count, datatype, dest, tag, comm, request)
 - MPI_Irecv(buffer, count, datatype, source, tag, comm, request)
 - Pour compléter ces opérations, on utilise MPI_Wait ou MPI_Test

4. Communications Collectives (8 points)

- Analysez le code suivant utilisant MPI_Bcast et expliquez son fonctionnement :

```

1      int main(int argc, char *argv[]) {
2          int rank, size, data;
3          MPI_Init(&argc, &argv);
4          MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5          MPI_Comm_size(MPI_COMM_WORLD, &size);
6
7          if (rank == 0) {
8              data = 42;
9          }
10
11         MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
12         printf("Process %d received data: %d\n", rank, data);
13
14         MPI_Finalize();
15         return 0;
16     }
17

```

Correction :

MPI_Bcast diffuse la valeur de data du processus 0 à tous les autres processus.

— Analysez le code suivant utilisant MPI_Reduce et expliquez son fonctionnement :

```

1      int main(int argc, char *argv[]) {
2          int rank, size, local_sum = rank, global_sum;
3          MPI_Init(&argc, &argv);
4          MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5          MPI_Comm_size(MPI_COMM_WORLD, &size);
6
7          MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);
8
9          if (rank == 0) {
10             printf("Global sum: %d\n", global_sum);
11         }
12
13         MPI_Finalize();
14         return 0;
15     }
16

```

Correction :

MPI_Reduce additionne les valeurs de local_sum de tous les processus et stocke le résultat dans global_sum du processus 0.

5. Comment MPI gère-t-il la synchronisation entre les processus ? Donnez un exemple de synchronisation explicite et implicite. (4 points)

Correction :

MPI utilise des barrières (MPI_Barrier) pour la synchronisation explicite. Les communications collectives comme MPI_Bcast impliquent une synchronisation implicite.

2 Exercices (30 points)

1. Écrivez un programme MPI en C ou C++ qui calcule la somme des éléments d'un tableau de taille n en utilisant p processus. Utilisez une communication collective pour réduire les résultats. (10 points)

Correction :

Utilisez MPI_Reduce pour additionner les sommes partielles calculées par chaque processus.

2. Implémentez un programme MPI en C ou C++ qui trouve le maximum d'un tableau de taille n en utilisant p processus. Utilisez une communication collective pour obtenir le résultat final. (10 points)

Correction :

Utilisez MPI_Reduce avec l'opérateur MPI_MAX pour trouver le maximum global.

3. Correction de Code (20 points)

— Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées :

```

1      int main(int argc, char *argv[]) {
2          int rank, size, data = rank;
3          MPI_Init(&argc, &argv);
4          MPI_Comm_rank(MPI_COMM_WORLD, &rank);

```

```

5         MPI_Comm_size(MPI_COMM_WORLD, &size);
6
7         MPI_Allreduce(&data, &data, 1, MPI_INT, MPI_SUM,
8 MPI_COMM_WORLD);
9         printf("Process %d has data: %d\n", rank, data);
10
11        MPI_Finalize();
12        return 0;
13    }

```

Correction :

Erreur : data est utilisé à la fois comme buffer d'envoi et de réception.
Correction : utiliser deux variables distinctes.

- Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées :

```

1         int main(int argc, char *argv[]) {
2             int rank, size, local_data = rank, gathered_data[10];
3             MPI_Init(&argc, &argv);
4             MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5             MPI_Comm_size(MPI_COMM_WORLD, &size);
6
7             MPI_Gather(&local_data, 1, MPI_INT, gathered_data, 1, MPI_INT
8 , 0, MPI_COMM_WORLD);
9
10            if (rank == 0) {
11                for (int i = 0; i < size; i++) {
12                    printf("Gathered data[%d]: %d\n", i, gathered_data[i
13                ]);
14            }
15
16            MPI_Finalize();
17            return 0;
18        }

```

Correction :

Erreur : gathered_data doit être de taille size. Correction : allouer dynamiquement gathered_data en fonction de size.

- Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées :

```

1         int main(int argc, char *argv[]) {
2             int rank, size, data;
3             MPI_Status status;
4             MPI_Init(&argc, &argv);
5             MPI_Comm_rank(MPI_COMM_WORLD, &rank);
6             MPI_Comm_size(MPI_COMM_WORLD, &size);
7
8             if (rank == 0) {
9                 data = 42;
10                MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
11            } else {
12                MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status
13            );
14                printf("Process %d received data: %d\n", rank, data);
15            }
16
17            MPI_Finalize();
18            return 0;
19        }

```

18

}

Correction :

Erreur : seul le processus 1 reçoit les données. Correction : utiliser une boucle pour envoyer les données à tous les processus.

- Corrigez le code suivant pour qu'il fonctionne correctement avec un nombre arbitraire de processus MPI. Expliquez les erreurs et les corrections apportées :

```

1      int main(int argc, char *argv[]) {
2          int rank, size, data_in, data_out;
3          MPI_Status status;
4          MPI_Init(&argc, &argv);
5          MPI_Comm_rank(MPI_COMM_WORLD, &rank);
6          MPI_Comm_size(MPI_COMM_WORLD, &size);
7
8          data_in = rank;
9          MPI_Send(&data_in, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
10         MPI_Recv(&data_out, 1, MPI_INT, size - 1, 0, MPI_COMM_WORLD,
11                 &status);
12
13         MPI_Finalize();
14         return 0;
15     }

```

Correction :

Erreur : Il y a un deadlock lié à l'utilisation des communications MPI bloquantes. Tous les processus vont être bloqués lors de la phase d'envoi du message.

```

38 #include <mpi.h>
39 #include <stdio.h>
40
41 int main(int argc, char *argv[]) {
42     int rank, size, data_in, data_out;
43     MPI_Status status;
44     MPI_Request request_send, request_recv;
45
46     MPI_Init(&argc, &argv);
47     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
48     MPI_Comm_size(MPI_COMM_WORLD, &size);
49
50     data_in = rank;
51
52     // Calculer le rang du processus suivant et précédent
53     int next_rank = (rank + 1) % size;
54     int prev_rank = (rank - 1 + size) % size;
55
56     // Envoyer un message au processus suivant (non bloquant)
57     MPI_Isend(&data_in, 1, MPI_INT, next_rank, 0,
58              MPI_COMM_WORLD, &request_send);
59
60     // Recevoir un message du processus précédent (non
61     // bloquant)
62     MPI_Irecv(&data_out, 1, MPI_INT, prev_rank, 0,
63              MPI_COMM_WORLD, &request_recv);
64
65     // Attendre la fin des communications
66     MPI_Wait(&request_send, MPI_STATUS_IGNORE);
67     MPI_Wait(&request_recv, &status);
68
69     // Afficher les résultats
70     printf("Processus %d a reçu %d du processus %d\n", rank,

```

```
data_out, prev_rank);  
68  
69 MPI_Finalize();  
70 return 0;  
71 }
```

3 Problème (50 points)

Contexte

Implémentez un programme qui distribue un tableau d'entiers entre plusieurs processus, calcule la moyenne de ces éléments de manière parallèle et envoie les résultats au processus principal pour calculer la moyenne totale. Vous disposez de p processus.

Questions

1. Analyse du Problème (10 points)

- Décrivez une approche naïve pour distribuer le tableau et calculer la moyenne en parallèle en utilisant MPI.

Correction :

Approche naïve : distribuer les éléments du tableau de manière égale entre les processus, calculer la somme locale, puis utiliser `MPI_Reduce` pour obtenir la somme totale et calculer la moyenne.

- Quelle est la complexité de cette approche en termes de temps et de communication ?

Correction :

Complexité en temps : $O(n/p)$ pour le calcul local et $O(\log p)$ pour la réduction. Complexité en communication : $O(p)$ messages.

2. Conception de l'Algorithme (15 points)

- Proposez une méthode plus efficace pour distribuer le tableau et calculer la moyenne en parallèle.

Correction :

Méthode efficace : utiliser `MPI_Scatter` pour distribuer les éléments du tableau et `MPI_Reduce` pour obtenir la somme totale.

- Expliquez comment vous répartissez les tâches entre les processus.

Correction :

Répartition des tâches : chaque processus reçoit une partie égale du tableau et calcule la somme locale.

- Quelles communications collectives utilisez-vous et pourquoi ?

Correction :

Communications collectives : `MPI_Scatter` pour la distribution et `MPI_Reduce` pour la réduction des sommes locales.

3. Implémentation (25 points)

- Écrivez le pseudo-code de votre algorithme parallèle pour le calcul de la moyenne.

Correction :

Pseudo-code :

- (a) Initialiser MPI et obtenir le rang et la taille du communicateur.
- (b) Si processus 0, lire le tableau et utiliser `MPI_Scatter` pour distribuer les éléments.
- (c) Chaque processus calcule la somme locale de ses éléments.
- (d) Utiliser `MPI_Reduce` pour obtenir la somme totale sur le processus 0.
- (e) Si processus 0, calculer la moyenne et afficher le résultat.

- Incluez les appels MPI nécessaires pour les communications.

Correction :

Appels MPI : `MPI_Scatter`, `MPI_Reduce`.

- Décrivez les étapes clés de l'implémentation, y compris l'initialisation, la répartition des données, le calcul local de la moyenne, et la fusion des résultats.

Correction :

Étapes clés :

- (a) Initialisation MPI.
- (b) Distribution des données avec `MPI_Scatter`.
- (c) Calcul local de la somme.
- (d) Réduction des sommes locales avec `MPI_Reduce`.
- (e) Calcul de la moyenne sur le processus 0.

- Écrivez le code C complet pour l'implémentation.

Correction :

Code C :

```

1      #include <mpi.h>
2      #include <stdio.h>
3      #include <stdlib.h>
4
5      int main(int argc, char *argv[]) {
6          int rank, size, n, *array = NULL, *
local_array = NULL, local_sum, global_sum;
7          MPI_Init(&argc, &argv);
8          MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9          MPI_Comm_size(MPI_COMM_WORLD, &size);
10
11         if (rank == 0) {
12             // Lire la taille du tableau et
allouer de la mémoire
13             printf("Entrez la taille du tableau :
");
14             scanf("%d", &n);
15             array = (int *)malloc(n * sizeof(int))
;
16             printf("Entrez les éléments du tableau
: ");
17             for (int i = 0; i < n; i++) {
18                 scanf("%d", &array[i]);
19             }

```

```
20     }
21
22     // Diffuser la taille du tableau à tous
les processus
23     MPI_Bcast(&n, 1, MPI_INT, 0,
MPI_COMM_WORLD);
24
25     // Allouer de la mémoire pour le sous-
tableau local
26     local_array = (int *)malloc((n / size) *
sizeof(int));
27
28     // Distribuer les éléments du tableau aux
processus
29     MPI_Scatter(array, n / size, MPI_INT,
local_array, n / size, MPI_INT, 0, MPI_COMM_WORLD);
30
31     // Calculer la somme locale
32     local_sum = 0;
33     for (int i = 0; i < n / size; i++) {
34         local_sum += local_array[i];
35     }
36
37     // Réduire les sommes locales pour obtenir
la somme globale
38     MPI_Reduce(&local_sum, &global_sum, 1,
MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
39
40     // Calculer et afficher la moyenne sur le
processus 0
41     if (rank == 0) {
42         double average = (double)global_sum /
n;
43         printf("La moyenne des éléments du
tableau est : %f\n", average);
44         free(array);
45     }
46
47     free(local_array);
48     MPI_Finalize();
49     return 0;
50 }
51
```